CSA1735 MS2-AI Lab Programs

PROGRAM 1: 8-Puzzle Problem (Breadth-First Search)

Register Number: 192417007

AIM

To solve the 8-puzzle by moving the blank space until the initial arrangement becomes the goal arrangement.

ALGORITHM

1. Store the initial and goal boards as tuples.
2. Put the initial board into a queue and mark it as visited.
3. Remove a board from the front of the queue. If it is the goal, stop.
4. Generate valid boards by swapping the blank (0) with an adjacent tile.
5. Add each unvisited board to the queue and remember its parent.
6. Reconstruct and print the path from the initial board to the goal.

PROGRAM

```
from collections import deque

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

moves = {
    0: [1,  3], 1: [0, 2, 4], 2: [1, 5],
    3: [0, 4, 6], 4: [1, 3, 5, 7], 5: [2, 4, 8],
    6: [3, 7], 7: [4, 6, 8], 8: [5, 7]
}

queue = deque([start])
parent = {start: None}

while queue:
    state = queue.popleft()
    if state == goal:
        break
    zero = state.index(0)
    for nxt in moves[zero]:
        board = list(state)
        board[zero], board[nxt] = board[nxt], board[zero]
        child = tuple(board)
        if child not in parent:
            parent[child] = state
            queue.append(child)

path = []
state = goal
while state is not None:
    path.append(state)
    state = parent[state]
path.reverse()

print("Solution steps:", len(path) - 1)
for board in path:
    for i in range(0, 9, 3):
        print(board[i:i + 3])
    print()
```

OUTPUT

```
Solution steps: 2
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

PROGRAM 2: 8-Queen Problem (Backtracking)

Register Number: 192417007

AIM

To place eight queens on an 8 × 8 chessboard so that no two queens attack each other.

ALGORITHM

1. Start with an empty chessboard represented by one column position per row.
2. Try placing a queen in each column of the current row.
3. Check that the column and both diagonals are safe.
4. If safe, place the queen and solve the next row recursively.
5. If no position works, remove the previous queen and try another position.
6. Print the board when queens have been placed in all eight rows.

PROGRAM

```
N = 8
board = [-1] * N

def safe(row, col):
    for r in range(row):
        if board[r] == col:
            return False
        if abs(board[r] - col) == abs(r - row):
            return False
    return True

def solve(row):
    if row == N:
        return True
    for col in range(N):
        if safe(row, col):
            board[row] = col
            if solve(row + 1):
                return True
            board[row] = -1
    return False

if solve(0):
    for row in range(N):
        print(" ".join("Q" if board[row] == col else "."
                       for col in range(N)))
else:
    print("No solution found")
```

OUTPUT

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

PROGRAM 3: Water Jug Problem (BFS)

Register Number: 192417007

AIM

To measure exactly two litres of water using a 4-litre jug and a 3-litre jug.

ALGORITHM

1. Represent a state as (amount in 4-litre jug, amount in 3-litre jug).
2. Start at (0, 0) and add it to a queue.
3. From each state, generate states by filling, emptying, or pouring between jugs.
4. Add new states to the queue if they have not been visited.
5. Stop when either jug contains exactly two litres.
6. Print the sequence of states leading to the goal.

PROGRAM

```
from collections import deque

start = (0, 0)
goal_amount = 2
queue = deque([start])
parent = {start: None}

def next_states(a, b):
    return [
        (4, b), (a, 3), (0, b), (a, 0),
        (a - min(a, 3 - b), b + min(a, 3 - b)),
        (a + min(b, 4 - a), b - min(b, 4 - a))
    ]

goal = None
while queue:
    state = queue.popleft()
    a, b = state
    if a == goal_amount or b == goal_amount:
        goal = state
        break
    for child in next_states(a, b):
        if child not in parent:
            parent[child] = state
            queue.append(child)

path = []
while goal is not None:
    path.append(goal)
    goal = parent[goal]
path.reverse()

for state in path:
    print(state)
```

OUTPUT

```
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

PROGRAM 4: Crypt-Arithmetic Problem (SEND + MORE = MONEY)

Register Number: 192417007

AIM

To find distinct digits for the letters in SEND + MORE = MONEY such that the addition is mathematically correct.

ALGORITHM

1. Assign digits to letters, ensuring that different letters have different digits.
2. The leading letters S and M cannot be zero.
3. Check each candidate assignment against SEND + MORE = MONEY.
4. Stop when a valid assignment is found and display the words as numbers.

PROGRAM

```
from itertools import permutations

letters = "SENDMORY"
digits = "0123456789"

for p in permutations(digits, len(letters)):
    d = dict(zip(letters, p))
    if d["S"] == "0" or d["M"] == "0":
        continue

    send = int("".join(d[x] for x in "SEND"))
    more = int("".join(d[x] for x in "MORE"))
    money = int("".join(d[x] for x in "MONEY"))

    if send + more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        break
```

OUTPUT

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

PROGRAM 5: Missionaries and Cannibals Problem (BFS)

Register Number: 192417007

AIM

To move three missionaries and three cannibals across a river safely using a boat that carries one or two people.

ALGORITHM

1. Represent each state as (missionaries left, cannibals left, boat side).
2. Start at (3, 3, 0), where 0 means the boat is on the left.
3. Generate all possible one- or two-person boat trips.
4. Reject states where cannibals outnumber missionaries on either bank when missionaries are present.
5. Use BFS and a visited set to find a safe path to (0, 0, 1).
6. Print the states in the solution path.

PROGRAM

```
from collections import deque

start = (3, 3, 0)  # 0 = left bank, 1 = right bank
goal = (0, 0, 1)
moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

def valid(m, c):
    mr, cr = 3 - m, 3 - c
    return (0 <= m <= 3 and 0 <= c <= 3 and
            (m == 0 or m >= c) and
            (mr == 0 or mr >= cr))

queue = deque([start])
parent = {start: None}

while queue:
    state = queue.popleft()
    if state == goal:
        break
    m, c, boat = state
    for dm, dc in moves:
        if boat == 0:
            child = (m - dm, c - dc, 1)
        else:
            child = (m + dm, c + dc, 0)
        if valid(child[0], child[1]) and child not in parent:
            parent[child] = state
            queue.append(child)

path = []
state = goal
while state is not None:
    path.append(state)
    state = parent[state]
path.reverse()
for state in path:
    print(state)
```

OUTPUT

```
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(0, 2, 0)
(0, 0, 1)
```

PROGRAM 6: Vacuum Cleaner Problem

Register Number: 192417007

AIM

To simulate a simple vacuum cleaner agent that cleans two rooms when they are dirty.

ALGORITHM

1. Represent the two rooms as either Dirty or Clean.
2. Start the vacuum cleaner in Room A.
3. If the current room is dirty, clean it.
4. Move to the other room and clean it if necessary.
5. Display the final condition of both rooms.

PROGRAM

```
rooms = {"A": "Dirty", "B": "Dirty"}
location = "A"

while True:
    if rooms[location] == "Dirty":
        print("Clean room", location)
        rooms[location] = "Clean"

    if location == "A":
        location = "B"
    else:
        break

print("Final room status:", rooms)
```

OUTPUT

```
Clean room A
Clean room B
Final room status: {'A': 'Clean', 'B': 'Clean'}
```

PROGRAM 7: Breadth-First Search (BFS)

Register Number: 192417007

AIM

To implement Breadth-First Search to visit graph nodes level by level from a starting node.

ALGORITHM

1. Create a queue containing the starting node.
2. Mark the starting node as visited.
3. Remove the front node from the queue and print it.
4. Add each unvisited neighbour to the queue and mark it visited.
5. Repeat until the queue is empty.

PROGRAM

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": ["F"], "F": []
}

def bfs(start):
    visited = {start}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        print(node, end=" ")
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

bfs("A")
```

OUTPUT

```
A B C D E F
```

PROGRAM 8: Depth-First Search (DFS)

Register Number: 192417007

AIM

To implement Depth-First Search to explore a graph by going as deep as possible before backtracking.

ALGORITHM

1. Start from the selected node and mark it visited.
2. Print the current node.
3. Visit an unvisited neighbour recursively.
4. Backtrack when the current node has no unvisited neighbours.
5. Continue until all reachable nodes are visited.

PROGRAM

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": ["F"], "F": []
}
visited = set()

def dfs(node):
    visited.add(node)
    print(node, end=" ")
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(neighbour)

dfs("A")
```

OUTPUT

```
A B D E F C
```

PROGRAM 9: Travelling Salesman Problem (Brute Force)

Register Number: 192417007

AIM

To find the shortest tour that visits every city once and returns to the starting city.

ALGORITHM

1. Store the distances between cities in a cost matrix.

2. Fix the first city as the starting city.

3. Generate all possible orders of the remaining cities.

4. Calculate the cost of each complete tour, including the return to the start.

5. Keep and print the tour with the smallest total cost.

PROGRAM

```
from itertools import permutations

cities = ["A", "B", "C", "D"]
cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

best_cost = float("inf")
best_route = None

for order in permutations(range(1, len(cities))):
    route = (0,) + order + (0,)
    total = sum(cost[route[i]][route[i + 1]]
                for i in range(len(route) - 1))
    if total < best_cost:
        best_cost = total
        best_route = route

print("Best tour:", " -> ".join(cities[i] for i in best_route))
print("Minimum cost:", best_cost)
```

OUTPUT

```
Best tour: A -> B -> D -> C -> A
Minimum cost: 80
```

PROGRAM 10: A* Search Algorithm

Register Number: 192417007

AIM

To implement A* search to find a minimum-cost path from a start node to a goal node using path cost and a heuristic.

ALGORITHM

1. Store each node's neighbours and edge costs in a graph.
2. Set g(start) = 0 and calculate f(n) = g(n) + h(n).
3. Put the start node in a priority queue ordered by the lowest f value.
4. Remove the lowest-f node. If it is the goal, reconstruct the path.
5. For each neighbour, update its cost and parent when a cheaper path is found.
6. Continue until the goal is reached or the queue becomes empty.

PROGRAM

```
import heapq

graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("C", 2), ("D", 5)],
    "C": [("D", 1)],
    "D": []
}
h = {"A": 4, "B": 3, "C": 1, "D": 0}

start, goal = "A", "D"
open_list = [(h[start], 0, start)]
parent = {start: None}
g_cost = {start: 0}

while open_list:
    f, g, node = heapq.heappop(open_list)
    if node == goal:
        break
    if g != g_cost[node]:
        continue
    for neighbour, edge_cost in graph[node]:
        new_g = g + edge_cost
        if new_g < g_cost.get(neighbour, float("inf")):
            g_cost[neighbour] = new_g
            parent[neighbour] = node
            heapq.heappush(open_list,
                           (new_g + h[neighbour], new_g, neighbour))

path = []
node = goal
while node is not None:
    path.append(node)
    node = parent[node]
path.reverse()
print("Path:", " -> ".join(path))
print("Total cost:", g_cost[goal])
```

OUTPUT

```
Path: A -> B -> C -> D
Total cost: 4
```